

DeepRAD51C — 구조 정보 Encoding 진행상황

RAD51C 변이 효과 예측 모델의 구조 feature 파트 / 작성일 2026-07-12

1. 프로젝트 한눈에

RAD51C(376 aa) 단백질의 변이가 기능에 미치는 영향을 예측하는 모델을 만드는 중이며, 그중 구조 정보를 벡터로 인코딩하는 부분을 이번에 구현했다. 다루는 변이는 SAV(단일 아미노산 치환, missense)와 in-frame indel(결실/삽입) 두 종류다. nonsense / frameshift / splice / intron / synonymous는 범위에서 제외했고, 이번 인코딩은 구조 feature만 사용한다(ESM 임베딩 / LLR/PLLR은 넣지 않음).

각 변이는 최종적으로 25차원 구조 feature 벡터로 표현된다:

블록	차원	내용	언제 채워지나
Block A	11	앵커 residue 한 점의 구조 (pLDDT, 2차구조, 표면노출, 기능자리 6곳까지 거리)	모든 변이 (indel=시작위치, SAV=치환위치)
Block B	11	indel '자르고 다시 잇는 사건'의 구조적 결과 (재봉합 기하 + 영향 구간 + 흉터 + 연결성)	indel만 (SAV는 전부 0)
one-hot	3	[sav, del, ins] 3-way 구분	모든 변이

Block B(11-dim) 세부 구성

그룹	dim	주요 feature
① 재봉합 기하	3	junction_strain, junction_orient_cos, seam_both_loop
② 영향 구간 (span pooling)	4	span_min2_rsasa, span_max2_plddt, span_frac_ss, span_min_site_dist
③ 흉터 자리	1	seam_mean_rsasa
④ 연결성	1	span_external_contacts
⑤ 크기	2	del_len_norm, ins_len_norm

2. 핵심 결과

2.1 데이터

scope 적용 후 총 4914개 변이: SAV 4555 / 결실 357 / 삽입 2. (split: train 3449 / val 723 / test 742, position 기반 분할 유지)

2.2 검증 — 결실에서 구조 feature가 실제 z_score와 상관있나

z_score가 낮을수록 기능 손상(depleted). '구조가 더 파괴적이면 z가 낮아진다'는 예측이 맞는지 확인.

feature	ρ (결실)	해석
A_plddt	-0.599	잘 접힌 core 결실 → 손상
B_seam_mean_rsasa	+0.501	표면 흉터 → 관대
B_span_external_contacts	-0.443	촘촘히 박힌 조각 결실 → 손상 (Block B 핵심)
B_span_min2_rsasa	+0.442	문힌 residue 결실 → 손상
B_junction_strain	+0.002	무신호 — 결실이 대부분 1-residue

2.3 구조-only 예측 baseline (Ridge 회귀, torch 없이)

구조 25-dim만으로 z_score를 예측. train으로 학습, 한 번도 안 본 test로 평가.

구분	n	Pearson r	Spearman ρ
train	3449	+0.480	+0.430
val	723	+0.502	+0.500

구분	n	Pearson r	Spearman ρ
test	742	+0.495	+0.431
test / SAV	689	+0.389	+0.351
test / 결실	52	+0.493	+0.430

분류(depleted vs unchanged): 같은 구조 예측 점수로 test AUC = 0.797 (n=740). → 구조 정보만으로도 기능 손상 변이를 꽤 잘 가려낸다.

3. 파일 안내 — 각 파일이 무엇인가

코드 파일 (code/)

파일	역할
block_b.py	핵심. build_block_a()로 구조 11-dim 테이블 계산, BlockBEncoder로 indel 11-dim 인코딩, encode_full()로 25-dim 조립.
assemble.py	조립기 데모. sav/del/ins 예시가 올바른 25-dim이 되는지 확인.
build_dataset.py	split_manifest.csv를 파싱해 전체 [4914,25] feature 표 생성.
analysis.py	구조 feature 진단 + Ridge 예측 baseline (분석 결과 JSON 저장).
generate_report.py	이 PDF 리포트 생성.
explain_blockb.py	조원 설명용 그림 PDF(Block_B_설명.pdf) 생성.

결과 파일 (results/)

파일	내용
rad51c_X.npy	모델 입력. [4914, 25] float 구조 feature 행렬.
rad51c_y.npy	라벨. [4914] z_score_D4_D14 (회귀 타깃).
rad51c_struct_features.csv	사람용 통합표. var_id / split / 라벨 + 25 feature.
rad51c_meta.csv	변이 메타 (var_id, split, type, 위치, 길이, 라벨).
rad51c_blockA.csv	residue별 Block A 12-dim 구조 테이블 (376행).
analysis_results.json	분석 수치 (상관 / baseline / AUC) 원본.

입력 파일 (inputs/)

파일	내용
AF-043502-F1.pdb	RAD51C AlphaFold WT 구조 (모든 구조 feature의 원천).
rad51c_residue_annotation.csv	기능 자리(Walker A/B, ATP, ssDNA, BCDX2/CX3) 정의 — 거리 계산용.
split_manifest.csv	SGE 실험 데이터: 변이 위치 / 타입 / 변이서열 / z_score / train/val/test 분할.
wt_sequence.txt	RAD51C WT 아미노산 서열 (376 aa, 검증용).

4. 설계 결정과 근거

- 구조만 사용 — 이 파트는 ESM 임베딩 / LLR과 독립적으로 구조 신호만 담당. 디버깅 / ablation이 깔끔.
- Block B = '재봉합 사건' 인코더(Direction 2) — indel은 SAV와 달리 사슬을 자르고 다시 잇는 사건. residue를 나열(pooling)하지 않고, 그 사건의 구조적 대가(끊긴 두 끝의 거리, 잘려나간 조각의 박힘 정도 등)를 인코딩. 가장 indel다운 정보를 컴팩트하게 담아 SAV=0 스킴과도 궁합이 좋음.
- SAV는 Block B를 0으로 — 치환은 점 하나(Block A)로 변이 전체가 설명되므로 추가 구조 정보가 없음. SAV/indel 구분은 바깥 one-hot이 담당.
- 삽입 feature 유지 — 이 데이터엔 삽입이 2개뿐이라 거의 0이지만, 향후 다른 유전자(삽입 많은 경우)로 확장할 때를 위해 일반 indel 인코더로 남겨둠.
- 정직한 한계 — junction_strain / junction_orient_cos는 이 데이터에서 무신호 (orient_cos 결실 $\rho = -0.045$). 결실이 거의 다 1-residue라 '재봉합 gap'이 변이마다 거의 안 변하기 때문. multi-residue 결실이 많은 유전자에선 살아날 feature라 유지.

5. 앞으로 하면 좋을 것 (우선순위 순)

할 일	설명
1. 다운스트림 MLP 연결	이 [N,25]을 문서 아키텍처의 구조 MLP 블록에 넣어 실제 예측까지. torch 필요(무거워짐, GPU 서버 권장).
2. 다른 feature 블록과 통합	ESM WT-difference 임베딩 등 다른 입력 블록과 concat하는 전체 파이프라인 조립.
3. junction_orient_cos 정리	이 데이터에서 무신호 — 유지하되, 최종 모델에서 ablation으로 기여도 재확인.
4. 외부 검증	Hu et al.(2023), Prakash et al.(2024)의 HR proficiency assay로 일반화 확인.
5. 다른 유전자 확장	BRCA-paralog, FANC 등 짧은 유전자로 framework 확장. 삼입 / multi-residue 결실 많은 데이터에서 Block B 진가 재확인.

6. 재현 방법

의존성: numpy, pandas, biotite (구조 인코딩) / reportlab (PDF). torch / scipy 불필요.

inputs/의 파일들을 작업 폴더에 두고 순서대로 실행:

```
python build_dataset.py    # split_manifest.csv → rad51c_X.npy, rad51c_y.npy, features.csv
python analysis.py         # → analysis_results.json (상관 / baseline / AUC)
python generate_report.py  # → 이 PDF
```

부록. 25개 feature 전체 — z_score Spearman 상관

전체 / SAV / 결실 각각에서의 상관. (SAV는 Block B가 0이라 B_* 상관은 정의되지 않음/0)

feature	전체 ρ	SAV ρ	결실 ρ
A_plddt	-0.334	-0.319	-0.599
A_ss_helix	-0.003	+0.004	-0.110
A_ss_sheet	-0.121	-0.118	-0.230
A_ss_loop	+0.096	+0.086	+0.281
A_rsasa	+0.327	+0.320	+0.441
A_dist_walker_a	+0.317	+0.316	+0.392
A_dist_walker_b	+0.318	+0.311	+0.497
A_dist_atp_contact	+0.297	+0.297	+0.379
A_dist_ssDNA_binding	+0.149	+0.145	+0.205
A_dist_bcdx2_interface	+0.218	+0.207	+0.406
A_dist_cx3_interface	+0.213	+0.200	+0.396
B_junction_strain	-0.226	+0.000	+0.002
B_junction_orient_cos	-0.114	+0.000	-0.045
B_seam_both_loop	-0.073	+0.000	+0.291
B_span_min2_rsasa	-0.202	+0.000	+0.442
B_span_max2_plddt	-0.233	+0.000	-0.599
B_span_frac_ss	-0.210	+0.000	-0.281
B_span_min_site_dist	-0.177	+0.000	+0.421
B_seam_mean_rsasa	-0.215	+0.000	+0.501
B_span_external_contacts	-0.234	+0.000	-0.443
B_del_len_norm	-0.229	+0.000	-0.044
B_ins_len_norm	+0.024	+0.000	+0.000
oh_sav	+0.226	+0.000	+0.000
oh_del	-0.229	+0.000	+0.000
oh_ins	+0.024	+0.000	+0.000